

Blossom Board

Collaborative Whiteboard Application

◆ Product Requirements Document & Implementation Plan ◆



Version:	1.0
Date:	May 2026
Track:	Frontend-Only (UI Simulation)
Hackathon:	HackStreet 2K26 — Problem Statement 4
Theme:	Cute Pink — Microsoft Word inspired aesthetic



❖ Table of Contents

#	Section	Pg
1	Executive Summary	3
2	Product Overview & Vision	3
3	User Personas	4
4	Feature Requirements (Pages)	4
4.1	Landing Page	4
4.2	Whiteboard Canvas	5
4.3	Task Manager (Notion-style)	7
5	UI / UX Design System	8
6	Non-Functional Requirements	9
7	Implementation Plan	9
7.1	Phase 1 — Foundation	9
7.2	Phase 2 — Canvas Core	10
7.3	Phase 3 — Sticky Notes & Collaboration UI	10
7.4	Phase 4 — Task Manager	10
7.5	Phase 5 — Landing Page & Polish	11
8	Tech Stack	11
9	Component Architecture	12
10	Risk & Mitigation	12
11	Success Metrics	13
12	Appendix — Keyboard Shortcuts	13

1. Executive Summary

Blossom Board is a **frontend-only collaborative whiteboard application** built for the HackStreet 2K26 hackathon (Problem Statement 4). It replicates the experience of professional digital whiteboards — Miro, FigJam, Notion — wrapped in a delightful **cute pink aesthetic** inspired by old-school Microsoft Word kawaii themes. The application is a **single-page React application** with three distinct screens: a feature-rich Landing Page, the main Whiteboard Canvas, and a Notion-calendar-style Task Manager. No backend is required — all state is managed in memory.

2. Product Overview & Vision

2.1 Vision Statement

"Make collaboration as cute and joyful as it is powerful — a digital canvas " where ideas bloom like cherry blossoms.

2.2 Problem Statement (verbatim scope)

Requirement Area	Description
Drawing Tools	Freehand drawing, geometric shapes, text input, sticky notes, colour selection
Element Interaction	Move, resize, layer elements with smooth interactions
Collaboration Simulation	Mock user cursors, participant avatars, activity indicators
Productivity Features	Zoom, pan, undo/redo, export as image or PDF
Performance	Fluid and intuitive — complex interactions without lag
Goal	Replicate tools like Miro using only frontend technologies

Select / Pointer	Click to select, drag to move element, Shift+click for multi-select	P0
Freehand Pen (—■)	Pressure-simulated stroke using quadratic bezier smoothing; colour picker + stroke width slider	P0
Eraser (■)	Erase pixels (canvas) or delete selected element	P0
Rectangle / Ellipse / Line / Arrow	Click-drag to draw; hold Shift to constrain to square/circle/45°	P0
Text Box (T)	Click to place editable <textarea> overlay; font size + bold/italic toggles	P0
Sticky Note (■)	Adds a draggable, resizable sticky note card with editable text; palette of 6 pastel colours	P0
Colour Picker	HEX input + 16-swatch grid; saved to recent colours	P0
Stroke Width Slider	Range 1–20 px; shows live preview	P0
Fill / No-fill toggle	For shapes; shows fill swatch	P1
Connector / Arrow tool	Draw arrows between elements; auto-route around shapes	P2
Image Upload (■■)	Drag-drop or click to upload PNG/JPG; placed as resizable element	P1
Laser Pointer	Temporary red dot that fades — for mock presentations	P2

4.2.2 Canvas Interactions

Interaction	Detail	Priority
Zoom In / Out	Mouse wheel scroll; Ctrl+= / Ctrl+-; zoom slider in bottom bar; range 10%–400%	P0
Pan / Navigate	Space + drag (hand tool); middle-mouse drag; trackpad two-finger pan	P0
Fit to Screen	Keyboard shortcut F or button; zooms to fit all content	P0
Undo / Redo	Ctrl+Z / Ctrl+Y; full command history stack (min. 50 steps); toast shows action name	P0
Select & Move	Drag selected elements; arrow keys for 1 px nudge; Shift+arrow for 10 px nudge	P0
Resize Handles	8-handle bounding box; corner handles maintain aspect ratio when Shift held	P0
Layer Order	Bring Forward, Send Back, Bring to Front, Send to Back via right-click menu	P1
Copy / Paste / Duplicate	Ctrl+C, Ctrl+V, Ctrl+D; pasted item offset by 10 px	P0
Group / Ungroup	Ctrl+G / Ctrl+Shift+G; grouped elements move together	P1
Right-click Context Menu	Delete, Duplicate, Lock, Copy Style, Add Link options	P1
Snap to Grid	Optional 10 px grid overlay; elements snap when within 5 px of gridline	P2
Mini-map	Bottom-right corner thumbnail showing current viewport position	P2

4.2.3 Sticky Notes (Detailed Spec)

Sticky notes are the signature feature of Blossom Board and should feel **delightful**.

- Pastel colour palette: Pink, Yellow, Mint, Lavender, Peach, Sky Blue
- Editable title + body text; auto-grows vertically up to max height

- Font: rounded sans-serif (e.g. Nunito); default size 14 px
- Drop shadow that intensifies on hover (CSS box-shadow transition)
- Fold/crinkle corner decoration (CSS clip-path or ::after pseudo element)
- Double-click to enter edit mode; click outside to confirm
- Emoji picker button in the top-right corner of each note
- Resize from bottom-right handle
- Rotate handle at top-centre (drag to rotate freely)
- Pin animation: note 'pops' in with a spring scale animation on creation
- Stack detection: semi-transparent overlay when notes overlap

4.2.4 Collaboration Simulation (Mock / UI Only)

- 3–4 mock cursors move around the canvas autonomously with different usernames shown as labels
- Participant avatar row in top-right: circular avatars with initials + status dot (green = active)
- Activity feed panel (right sidebar): timestamped log — 'Alice added a sticky note · 2s ago'
- Typing indicator on sticky notes: '...' dots appear on notes being 'edited' by mock users
- Join/leave toast notifications with avatar and name
- Cursor colours match each participant's avatar accent colour

4.2.5 Export Features

- Export as PNG — uses html2canvas to capture full board at current zoom or fitted
- Export as PDF — jsPDF renders canvas content with page tiling for large boards
- Export selection only — export only selected elements
- Copy to clipboard — copies viewport as PNG to clipboard (Clipboard API)

4.3 Page 3 — Task Manager (Notion / Calendar Style)

A dedicated task organisation screen inspired by **Notion database views** and **calendar boards**, keeping the same pink aesthetic.

Feature	Description	Priority
Kanban Board View	Columns: Backlog, In Progress, Review, Done. Drag-and-drop cards between columns.	P0
Calendar View	Monthly grid; tasks appear as pink pill-shaped chips on their due date. Click to open detail.	P0
List View	Table of tasks with sortable columns: Title, Assignee, Status, Priority, Due Date	P1
Task Card	Title, description, assignee avatar, priority badge (■ Low / ■ Medium / ■ High), due date, tags, checkboxes	P1
Add Task	Click + button or press N; modal with all fields; animated slide-in	P0
Task Detail Modal	Full-screen modal with rich text description area, comments section (nick), attachments	P1
Filters & Search	Filter by assignee, priority, tag; full-text search across titles	P1
Assignee Mock Users	Same 4 mock users from the collaboration simulation	P1
Quick Link to Board	Any task card has a 'View on Board' button that switches to canvas and highlights the linked sticky	P2
Dark Mode Toggle	Switches to a deep rose / charcoal dark theme while preserving pink accents	P2

5. UI / UX Design System



5.1 Colour Palette

Name	Hex	Usage
Deep Pink (Primary)	#E91E8C	CTAs, headings, active tool highlights
Mid Pink	#F48FB1	Borders, dividers, secondary buttons
Light Pink	#FCE4EC	Card backgrounds, input fields
Pale Pink	#FFF0F5	Page background, canvas background
Pink Accent	#FF80AB	Hover states, badges, tags
Dark Text	#3D1A2E	Body copy on light backgrounds
White	#FFFFFF	Text on dark backgrounds, canvas

5.2 Typography

Element	Font	Size	Weight
App Name / H1	Nunito	32–48 px	800 ExtraBold
Section Headings H2	Nunito	22–28 px	700 Bold
Sub-headings H3	Nunito	16–18 px	600 SemiBold
Body / Labels	Inter	13–15 px	400 Regular
Sticky Note Text	Nunito	14 px	500 Medium
Code / Mono	JetBrains Mono	12 px	400
Emoji / Decorative	System Emoji	—	—

5.3 Component Design Tokens

- Border radius: 12 px (cards), 8 px (buttons), 50 % (pills/avatars)
- Box shadow: 0 4px 16px rgba(233,30,140,0.15) for cards; 0 8px 32px rgba(233,30,140,0.25) on hover
- Transitions: all 0.2s ease for hover; spring animation for note creation (framer-motion)
- Toolbar: fixed left sidebar, 60 px wide, icons 28 px, active state: pink background pill
- Scrollbars: custom thin pink scrollbar via ::-webkit-scrollbar
- Cursor: custom heart cursor PNG when pen tool is active
- Focus ring: 2 px dashed #E91E8C outline for accessibility

6. Non-Functional Requirements

Category	Requirement
Performance	Canvas redraws at ≥ 60 fps during pan/zoom on 1080p. Freehand drawing latency < 16 ms.
Responsiveness	Full functionality on 1280 px+ wide screens. Tablet-friendly at 768 px.
Accessibility	ARIA labels on all toolbar buttons. Keyboard navigation for task manager. WCAG AA contrast for text.
Browser Support	Chrome 110+, Edge 110+, Firefox 115+, Safari 16+
Bundle Size	Initial JS bundle ≤ 500 KB gzipped. Code-split task manager and landing page.
State Persistence	Board state saved to localStorage every 10 s; restored on page refresh.
No Backend	Zero API calls to any server (export via client-side libraries only).

7. Implementation Plan

The project is split into **5 phases** designed for a hackathon timeline (approx. 24–36 hours of build time). Each phase has clear deliverables and a time estimate.

7.1 Phase 1 — Foundation (2–3 hrs)

- Scaffold React + Vite project with TypeScript
- Install: Tailwind CSS, Framer Motion, Zustand (state), jsPDF, html2canvas, react-dnd
- Set up global CSS design tokens (colours, fonts, scrollbar)
- Create Router with three routes: / /board /tasks
- Build persistent NavBar component with page links and pink logo
- Set up Zustand stores: boardStore (elements, history), taskStore (tasks)
- Implement auto-save to localStorage middleware

7.2 Phase 2 — Canvas Core (6–8 hrs)

- CanvasBoard component: HTML5 with React ref
- Implement viewport transform: pan offset + scale matrix; apply via ctx.setTransform
- Zoom handler: wheel event $\rightarrow$ clamp scale 0.1–4.0 $\rightarrow$ re-render
- Pan handler: space+mousedown drag / middle-mouse drag
- Element data model: { id, type, x, y, w, h, rotation, zIndex, style, content }
- Render loop: useEffect + requestAnimationFrame; clear and redraw all elements each frame
- Freehand pen: collect points on mousemove $\rightarrow$ smooth with cardinal spline $\rightarrow$ draw path

- Eraser: path collision detection on canvas pixel level
- Shape tools: Rectangle, Ellipse, Line, Arrow — mouse-down to mouse-up drag
- Text box: overlay positioned on canvas coordinates
- Selection tool: click-to-select (bounding box hit test), drag to multi-select
- Resize handles: 8 SVG circles over selected element bounding box
- Move: mousedown on selected element → drag → update x/y in store
- Undo/redo: command pattern — every mutation pushes to history stack
- Copy/paste/duplicate: store clipboard in Zustand; paste offset +10 px
- Layer controls: zIndex manipulation in store
- Fit-to-screen: calculate bounding box of all elements → set viewport

7.3 Phase 3 — Sticky Notes & Collaboration UI (4–5 hrs)

- StickyNote component: absolutely positioned div over canvas; Framer Motion spring on mount
- Colour picker for sticky notes: 6 presets + custom HEX input
- Editable text: contenteditable div with auto-height; emoji picker (emoji-mart library)
- Rotate handle: drag on top-centre circle → update element.rotation
- Fold corner: CSS ::after pseudo element with triangle clip-path
- MockCursor component: 4 instances with different names; move via setInterval + random waypoint animation
- AvatarRow component: top-right panel; 4 circular avatars with status dots
- ActivityFeed component: right sidebar; push events from boardStore mutations
- Toast notifications: join/leave events with framer-motion slide-in-out
- TypingIndicator: setInterval randomly assigns 'typing' state to mock users on notes
- Colour picker toolbar: HEX input, 16 swatches, recent colours row
- Stroke width slider: range input with live preview dot
- Image upload: FileReader → draw image on canvas as element
- Export PNG: html2canvas on board wrapper div
- Export PDF: jsPDF with canvas.toDataURL added as image
- Mini-map: small canvas in bottom-right reflecting full board at 1:20 scale

7.4 Phase 4 — Task Manager (4–5 hrs)

- TaskStore: tasks[], columns[], filters state in Zustand
- KanbanBoard: 4 columns (Tailwind flex); react-dnd DragDropContext for card movement
- TaskCard: pink card with title, avatar, priority badge, due date chip
- AddTask modal: form with all fields; slide-in animation
- TaskDetail modal: full-screen overlay; rich text area (simple markdown preview)
- CalendarView: 7×5 monthly grid built with date-fns; tasks rendered as pink pills on date cells
- ListView: sortable table using custom sort hook; alternating pink/white rows

- Filter panel: checkboxes for priority + assignee; text search input
- View switcher: Kanban / Calendar / List toggle buttons in top bar
- Mock assignees: same 4 users as collaboration simulation
- Priority badges: emoji + colour coded (■ Low = mint, ■ Medium = yellow, ■ High = deep pink)

7.5 Phase 5 — Landing Page & Polish (3–4 hrs)

- LandingPage: full-page scroll with Framer Motion scroll-triggered animations
- HeroSection: animated floating hearts (CSS keyframes), gradient background, CTA button
- FeatureGrid: 6 cards with icon, title, description; hover: scale + deeper shadow
- DemoPreview: looped CSS animation simulating drawing on a mini canvas thumbnail
- StatsRow: animated counters using react-countup
- Footer: pink gradient with heart decorations
- Global polish: audit all hover states, focus rings, transition timings
- Keyboard shortcuts: implement all shortcuts listed in Appendix
- Responsive audit: test at 768 px, 1024 px, 1440 px
- Performance pass: memoize heavy components; verify 60 fps on canvas
- Accessibility pass: add aria-labels; test keyboard navigation
- Final testing: cross-browser check on Chrome + Edge + Firefox

8. Tech Stack

Category	Library / Tool	Purpose
Framework	React 18 + Vite	UI components, fast HMR dev server
Language	TypeScript	Type safety across all components
State Management	Zustand	Lightweight global state for board & tasks
Styling	Tailwind CSS + CSS Modules	Utility-first styling + scoped animations
Animation	Framer Motion	Spring animations, gesture handlers, layout animations
Canvas Rendering	HTML5 Canvas API	Core drawing surface; raw 2D context
Drag & Drop	react-dnd	Kanban card drag-drop; element drag fallback
Date Handling	date-fns	Calendar view date calculations
Emoji Picker	emoji-mart	Emoji picker in sticky notes
Export PNG	html2canvas	Full-board screenshot to PNG
Export PDF	jsPDF	Canvas to PDF tile export
Routing	React Router v6	Three-page navigation
Icons	Lucide React	Consistent icon set for toolbar
Fonts	Google Fonts (Nunito, Inter)	Loaded via @import in CSS
Linting	ESLint + Prettier	Code quality
Build	Vite	Production bundle with code splitting

9. Component Architecture

Component / Module	Responsibility
App.tsx	Router, global providers (DnD, ThemeContext)
stores/boardStore.ts	Zustand: elements[], history, viewport, tool, selection
stores/taskStore.ts	Zustand: tasks[], columns[], filters, view
pages/LandingPage/	Hero, FeatureGrid, DemoPreview, Stats, Footer
pages/Board/	CanvasBoard, Toolbar, PropertiesPanel, CollabUI, MiniMap
pages/Tasks/	KanbanBoard, CalendarView, ListView, TaskModal, FilterPanel

components/canvas/	useDraw, useShape, usePen, useSelect, useTransform hooks
components/ui/StickyNote	Draggable, resizable, editable sticky note with emoji
components/ui/ColourPicker	Hex input + swatches + recent colours
components/collab/MockCursors	Animated mock cursors via interval
components/collab/AvatarRow	Participant bar + activity feed
components/collab/ActivityFeed	Right-panel event log
utils/exportUtils.ts	html2canvas + jsPDF export helpers
utils/historyUtils.ts	Command pattern push/pop/redo helpers
utils/geometryUtils.ts	Hit-testing, bounding boxes, rotation math
constants/theme.ts	Colour tokens, font sizes, spacing scales

10. Risk & Mitigation

Risk	Likelihood	Mitigation
Canvas perf below 60 fps on large boards	Medium	Limit elements to 500; use offscreen canvas for static layers; throttle freehand
html2canvas misses CSS styling	High	Test export early; fall back to canvas.toDataURL for pure-canvas areas
react-dnd conflicts with canvas mouse events	Medium	Use pointer-events: none on canvas when DnD is active; separate event tar
Emoji-mart bundle size	Low	Lazy-load emoji picker chunk; use dynamic import()
Framer Motion animation jank on low-end devices	Low	Add prefers-reduced-motion media query; skip spring on older devices
LocalStorage quota exceeded	Low	Compress state with LZ-String; warn user at 80% quota
Time overrun on hackathon timeline	Medium	P2 features are optional; core P0/P1 always ship first

11. Success Metrics

- All P0 features functional: freehand draw, shapes, sticky notes, zoom/pan, undo/redo, export
- Canvas maintains ≥ 60 fps during continuous freehand drawing on 1080p
- Sticky note creation-to-edit flow completes in ≤ 2 clicks
- Landing page Lighthouse score ≥ 90 Performance, ≥ 85 Accessibility
- Task manager Kanban drag-and-drop feels smooth (no jank) on Chrome + Edge
- Full keyboard shortcut set implemented and documented
- Export PNG produces correct output of the visible board area
- Zero console errors in production build
- 'Wow factor' — judges react positively to the aesthetic within 10 seconds of seeing it

12. Appendix — Keyboard Shortcuts

Shortcut	Action
V	Select tool
P / B	Pen / Brush tool
E	Eraser
R	Rectangle

O	Ellipse
L	Line
A	Arrow
T	Text box
S	Sticky note
I	Image upload
Ctrl + Z	Undo
Ctrl + Y / Ctrl+Shift+Z	Redo
Ctrl + C	Copy selected
Ctrl + V	Paste
Ctrl + D	Duplicate selected
Ctrl + A	Select all
Delete / Backspace	Delete selected
Ctrl + G	Group selection
Ctrl + Shift + G	Ungroup
Ctrl +]	Bring forward
Ctrl + [	Send backward
F	Fit board to screen
Space + Drag	Pan canvas
Ctrl + Scroll	Zoom in / out
Ctrl + 0	Reset zoom to 100%
Ctrl + Shift + E	Export PNG
N	New task (in Task Manager)
Escape	Deselect / close modal



◆ End of Document ◆ Blossom Board PRD v1.0 ◆ HackStreet 2K26 ◆

Made with  for the hackathon